



## Il Ritorno delle Pizze (Versione Hard) (fattorino\_hard)

È l'ora di pranzo allo stage per le Olimpiadi di Informatica e tutti gli studenti sono affamati! Tommaso aveva promesso di preparare la pizza per tutti, ma come al solito è troppo lento. Samuele, stufo di aspettare, decide di andare lui stesso in pizzeria a prendere le pizze.

La città è composta da  $N$  incroci, numerati da  $0$  a  $N - 1$ . Gli incroci sono collegati da  $M$  strade bidirezionali. La  $i$ -esima strada collega gli incroci  $a_i$  e  $b_i$  e richiede  $t_i$  minuti per essere percorsa.

Samuele intanto si è perso a Udine e, dato che si è fatto tardi, decide di tornare al Toppo, che si trova all'incrocio  $T$ .



Figura 1: Samuele che corre verso il Toppo.

In  $K$  incroci  $C_0, \dots, C_{K-1}$  di Udine ci sono dei chioschi che vendono tranci di pizza. Il chiosco nell'incrocio  $C_i$  vende un trancio di pizza con un livello di soddisfazione  $S_i$  (misurato in «quanto vale la pena fermarsi»).

Samuele è disposto a fermarsi al massimo in un chiosco lungo il suo percorso verso il Toppo, ma solo se il tempo extra aggiunto al suo percorso è al massimo uguale al livello di soddisfazione della pizza che prenderebbe. In altre parole, se fermarsi a prendere una pizza gli fa perdere  $t$  minuti in più rispetto al percorso più veloce, lo farà solo se il livello di soddisfazione è almeno  $t$ .

Dato che non sappiamo dove si trova Samuele, determina per ogni possibile incrocio iniziale se Samuele si fermerà a prendere la pizza oppure no.

### Dati di input

La prima riga contiene gli interi  $N$ ,  $M$ ,  $K$ ,  $T$ , rispettivamente il numero di incroci, il numero di strade, il numero di chioschi e l'incrocio del Toppo.

La  $1 + i$ -esima riga ( $0 \leq i < M$ ) contiene tre interi  $a_i$ ,  $b_i$  e  $t_i$ , che descrivono la presenza di una strada tra gli incroci  $a_i$  e  $b_i$ , con tempo di percorrimeto di  $t_i$  minuti.

La  $1 + M + i$ -esima riga ( $0 \leq i < K$ ) riga contiene gli interi  $C_i$  e  $S_i$ , rispettivamente l'incrocio dell' $i$ -esimo chiosco e il suo livello di soddisfazione.



Tra gli allegati a questo task troverai un template `pizzaiolo_hard.*` con un esempio di implementazione.

## Dati di output

Stampa  $N$  righe. All' $i$ -esima riga stampa 1 se Samuele, partendo dall' $i$ -esimo incrocio, si fermerà a prendere la pizza. Stampa 0 altrimenti.

## Assunzioni

- $2 \leq N \leq 50000$ .
- $1 \leq M \leq 100000$ .
- $1 \leq K \leq N$ .
- $0 \leq T < N$ .
- $0 \leq a_i, b_i < N$  e  $a_i \neq b_i$  per ogni  $0 \leq i < M$ .
- $1 \leq t_i \leq 10000$  per ogni  $0 \leq i < M$ .
- $0 \leq C_i < N$  per ogni  $0 \leq i < K$ .
- $1 \leq S_i \leq 10^9$  per ogni  $0 \leq i < K$ .
- Ogni incrocio è raggiungibile da ogni altro incrocio.
- Le strade sono bidirezionali.

## Assegnazione del punteggio

Il tuo programma verrà testato su diversi testcase raggruppati in subtask. Per ottenere il punteggio relativo ad un subtask, è necessario risolvere correttamente tutti i test che lo compongono.

- **Subtask 1** (0 punti)      Casi d'esempio.  
★★★★★
- **Subtask 2** (10 punti)       $N \leq 500$ .  
★★★☆☆
- **Subtask 3** (15 punti)       $K = 1$ .  
★★★☆☆
- **Subtask 4** (20 punti)       $N \leq 5000, M \leq 10000$ .  
★★★☆☆
- **Subtask 5** (25 punti)       $t_i = 1$  per ogni  $0 \leq i < M$ .  
★★★★☆
- **Subtask 6** (30 punti)      Nessuna limitazione aggiuntiva.  
★★★★★

## Esempi di input/output

| stdin                                     | stdout      |
|-------------------------------------------|-------------|
| 3 3 1 2<br>0 1 2<br>0 2 1<br>2 1 2<br>1 3 | 1<br>1<br>0 |
| 2 1 1 1<br>0 1 2<br>0 5                   | 1<br>1      |

## Spiegazione

Nel **primo caso d'esempio**:

- Se Samuele partisse dall'incrocio 0, compierebbe il percorso  $0 \rightarrow 1 \rightarrow 2$ , prendendo la pizza all'incrocio 1. Il percorso ottimale peggiora di 3 minuti. Dato che il livello di soddisfazione della pizza è 3, il percorso è valido.

- Se Samuele partisse dall'incrocio 1, compierebbe il percorso  $1 \rightarrow 2$ . Il percorso ottimale contiene già l'incrocio della pizza.
- Se Samuele partisse dall'incrocio 2, qualsiasi percorso che contiene l'incrocio 1 richiede almeno 4 minuti. Quindi non prenderebbe la pizza.

Nel **secondo caso d'esempio**:

- Se Samuele partisse dall'incrocio 0, compierebbe il percorso  $0 \rightarrow 1$ , prendendo la pizza.
- Se Samuele partisse dall'incrocio 1, compierebbe il percorso  $1 \rightarrow 0 \rightarrow 1$ , prendendo la pizza.